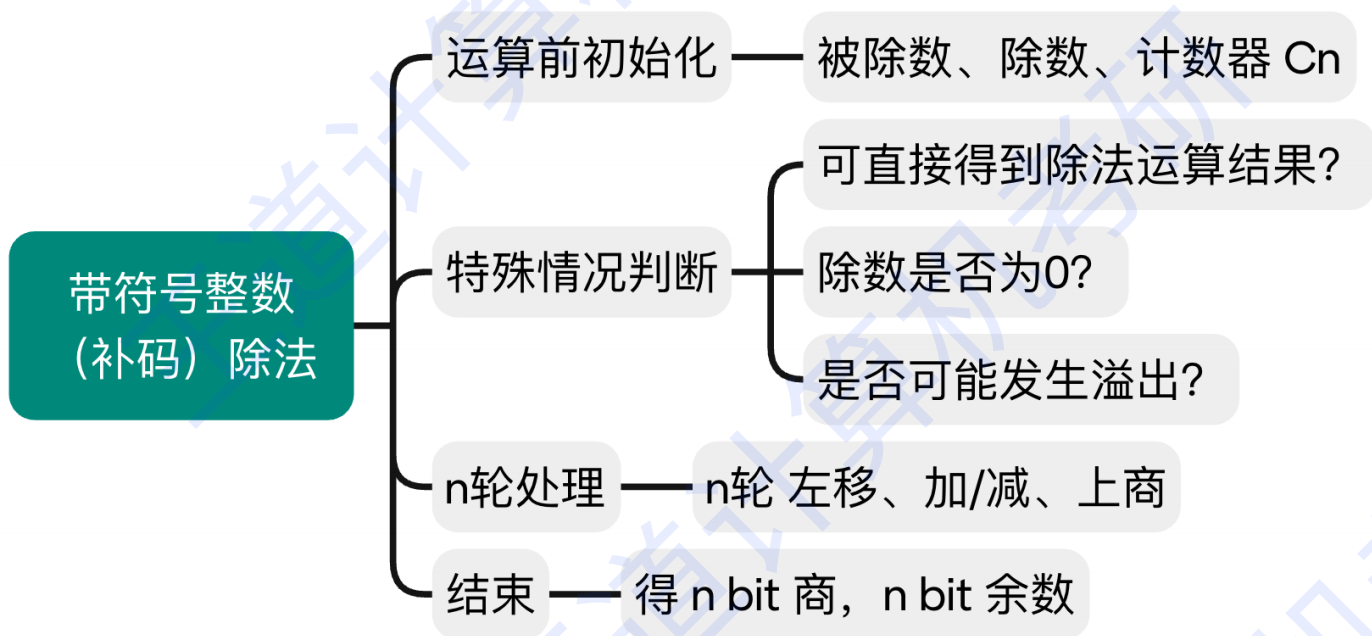


本节内容

带符号整数
(补码)

除法运算原理

知识总览

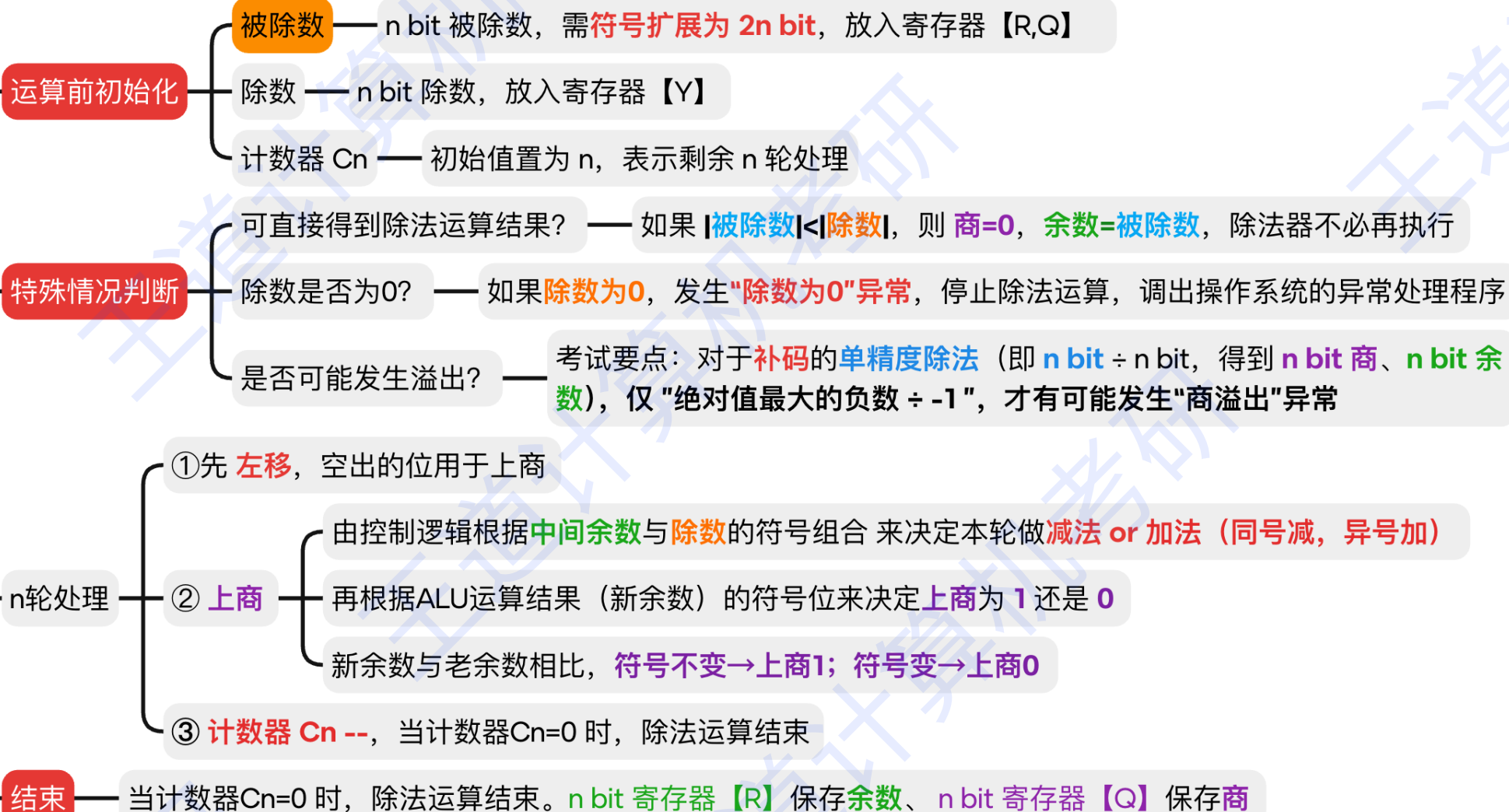


对于除法电路，建议**重点关注**：除法电路的**开始状态**、**结束状态**、**除法异常判断**



考点总结

带符号整数 (补码) 除法



例1：带符号整数除法

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

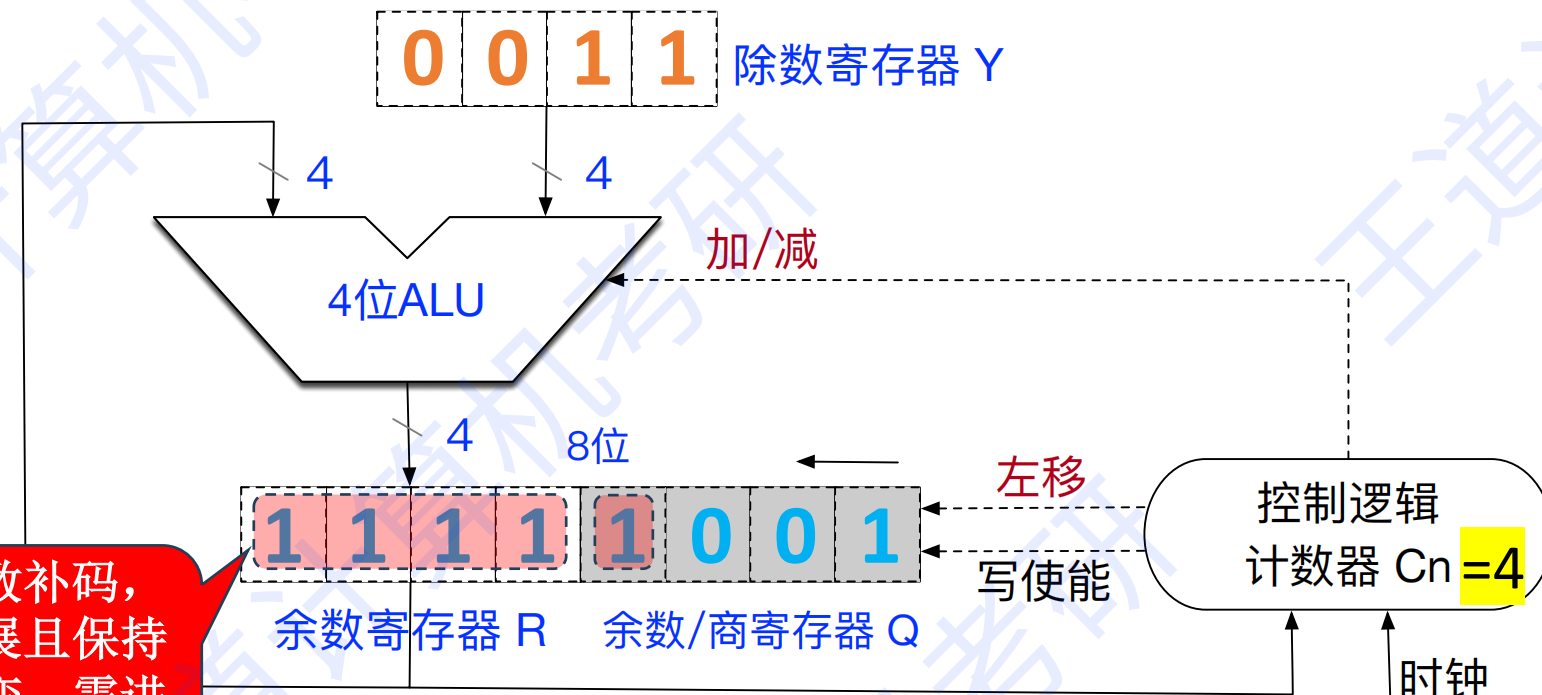
1001 $\rightarrow$ 被除数 $X=-7$

0011 $\rightarrow$ 除数 $Y=3$

1110 $\rightarrow$ 商 $Q=-2$

1111 $\rightarrow$ 余数 $R=-1$

带符号数补码，
位数扩展且保持
真值不变，需进
行“符号扩展”



运算前初始化

被除数 — n bit 被除数，需符号扩展为 $2n$ bit，放入寄存器 $[R, Q]$

除数 — n bit 除数，放入寄存器 $[Y]$

计数器 C_n — 初始值置为 n ，表示剩余 n 轮处理

例1：带符号整数除法

若被除数不足 2n bit，需要扩展为 2n bit，统一为 2n bit ÷ nbit

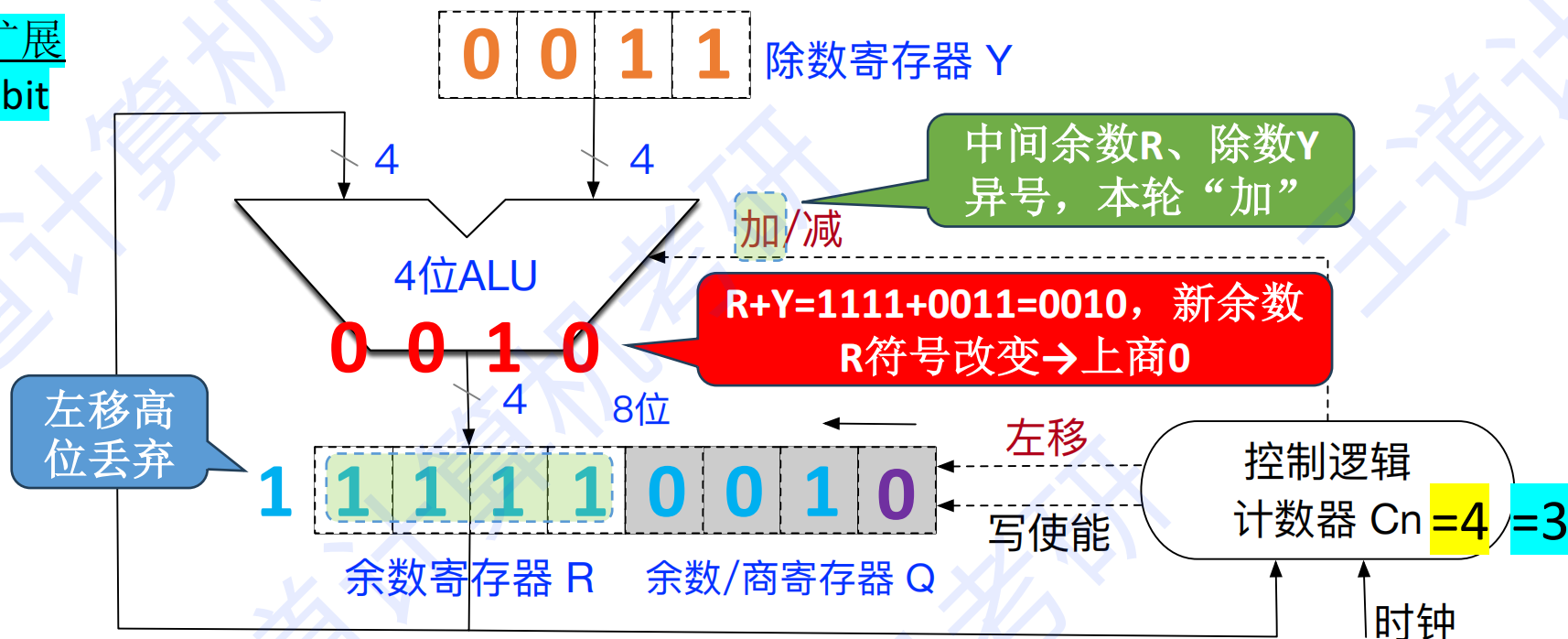
该除法器支持 2n bit ÷ n bit
最终得到 n bit 商、n bit 余数

1001 → 被除数 X = -7

0011 → 除数 Y = 3

1110 → 商 Q = -2

1111 → 余数 R = -1



- ① 先左移，空出的位用于上商
 - ② 上商
 - 由控制逻辑根据中间余数与除数的符号组合 来决定本轮做减法 or 加法（同号减，异号加）
 - 再根据ALU运算结果（新余数）的符号位来决定上商为 1 还是 0
 - 新余数与老余数相比，符号不变→上商1；符号变→上商0
 - ③ 计数器 Cn --，当计数器 Cn=0 时，除法运算结束
- n轮处理

例1: 带符号整数除法

若被除数不足 $2n$ bit, 需要扩展为 $2n$ bit, 统一为 $2n$ bit $\div$ n bit

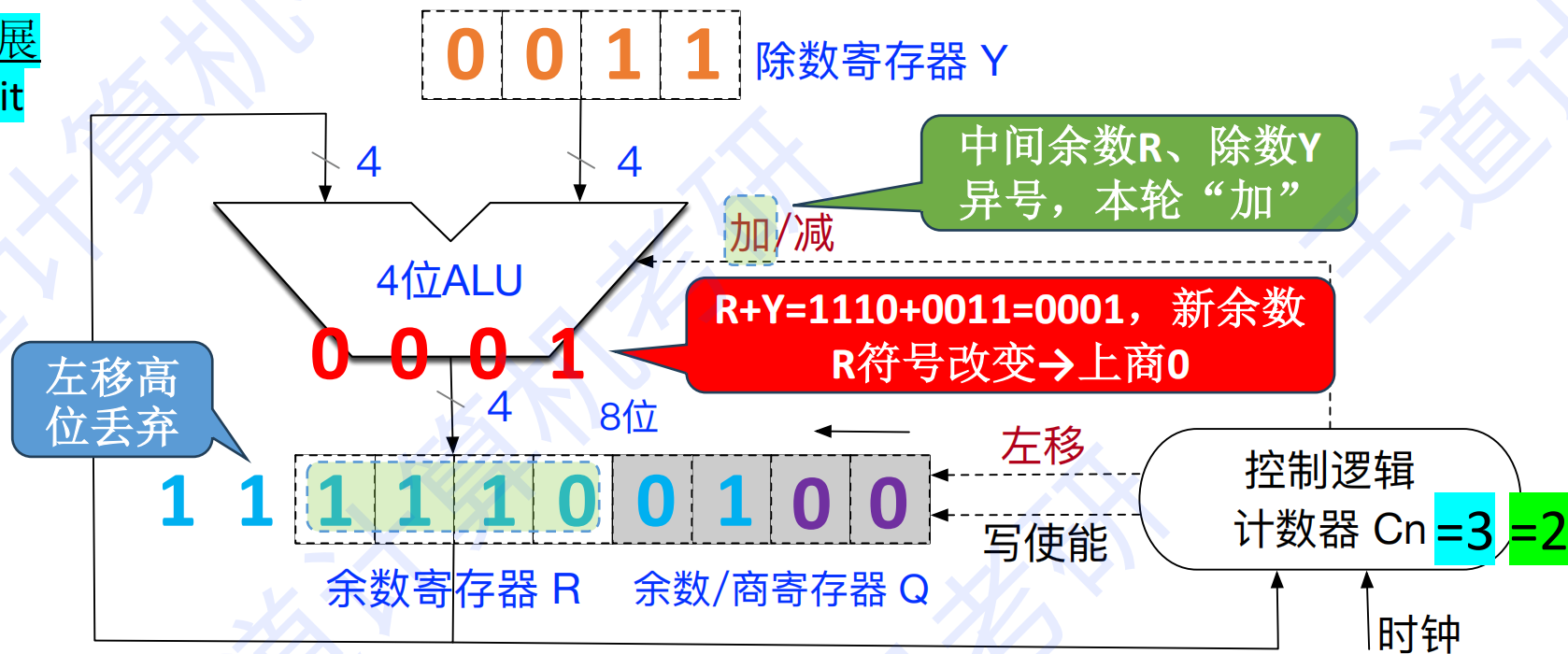
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

1001 $\rightarrow$ 被除数 $X=-7$

0011 $\rightarrow$ 除数 $Y=3$

1110 $\rightarrow$ 商 $Q=-2$

1111 $\rightarrow$ 余数 $R=-1$



- ①先左移, 空出的位用于上商
 - ②上商
 - 由控制逻辑根据中间余数与除数的符号组合 来决定本轮做减法 or 加法 (同号减, 异号加)
 - 再根据ALU运算结果 (新余数) 的符号位来决定上商为 1 还是 0
 - 新余数与老余数相比, 符号不变 $\rightarrow$ 上商1; 符号变 $\rightarrow$ 上商0
 - ③计数器 C_n-- , 当计数器 $C_n=0$ 时, 除法运算结束
- n轮处理

例1：带符号整数除法

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

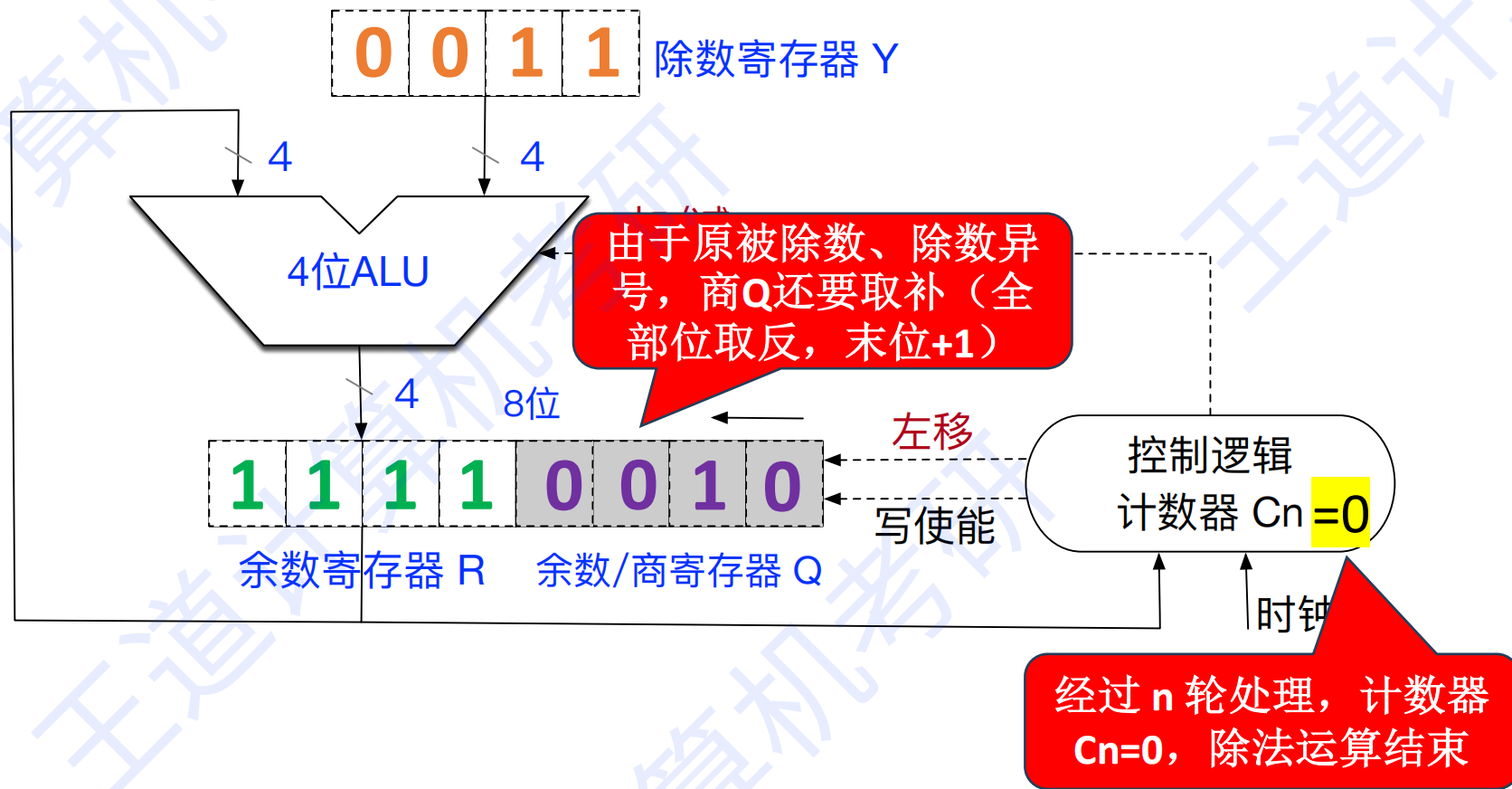
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

1001 $\rightarrow$ 被除数 $X=-7$

0011 $\rightarrow$ 除数 $Y=3$

1110 $\rightarrow$ 商 $Q=-2$

1111 $\rightarrow$ 余数 $R=-1$



结束——当计数器 $C_n=0$ 时，除法运算结束。 n bit 寄存器 **【R】** 保存余数、 n bit 寄存器 **【Q】** 保存商

例1：带符号整数除法

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

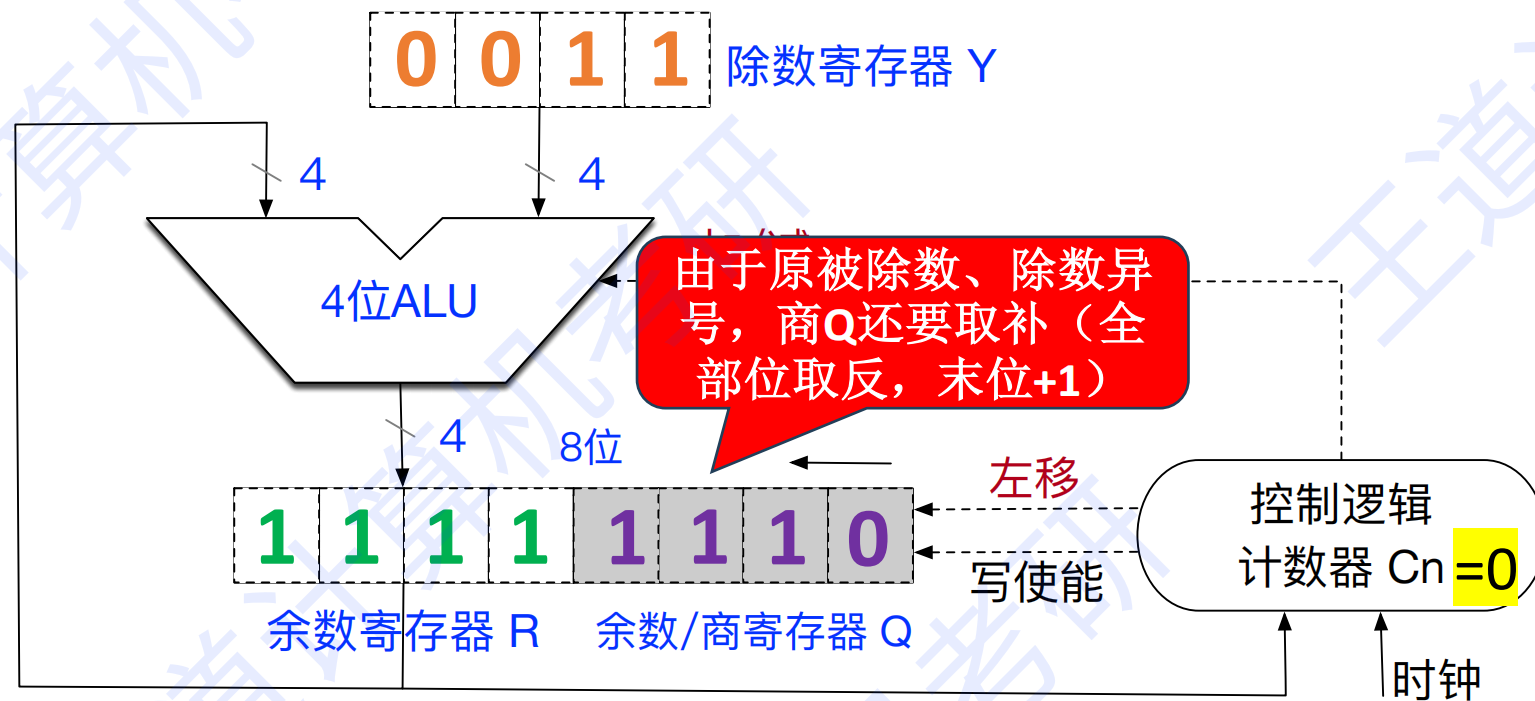
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

1001 $\rightarrow$ 被除数 $X=-7$

0011 $\rightarrow$ 除数 $Y=3$

1110 $\rightarrow$ 商 $Q=-2$

1111 $\rightarrow$ 余数 $R=-1$



结束——当计数器 $C_n=0$ 时，除法运算结束。 n bit 寄存器 **[R]** 保存余数、 n bit 寄存器 **[Q]** 保存商

例2：带符号整数除法

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

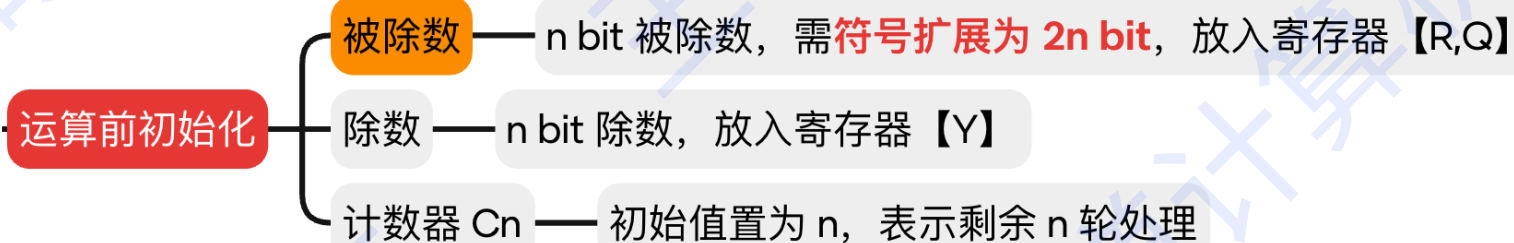
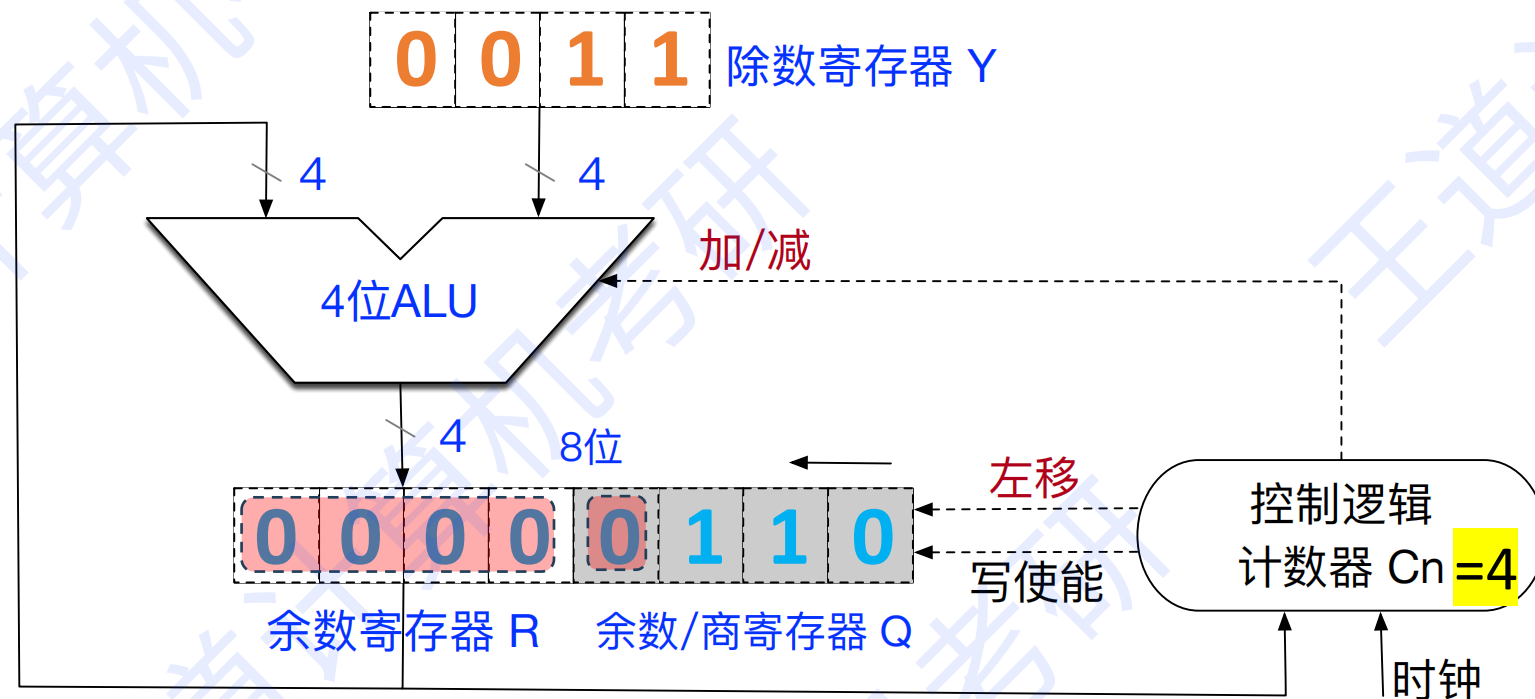
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

0110 $\rightarrow$ 被除数 $X=6$

0011 $\rightarrow$ 除数 $Y=3$

0010 $\rightarrow$ 商 $Q=2$

0000 $\rightarrow$ 余数 $R=0$



例2：带符号整数除法

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

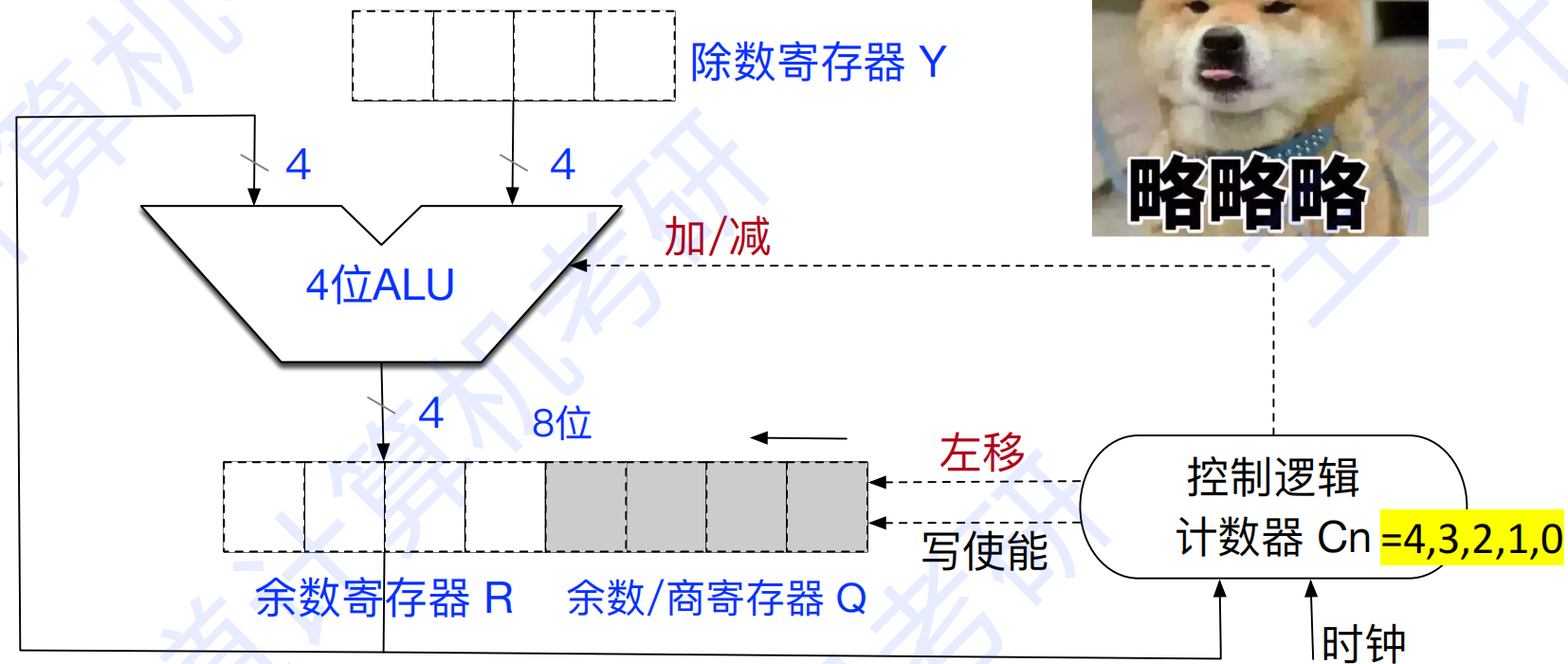
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

0110 $\rightarrow$ 被除数 $X=6$

0011 $\rightarrow$ 除数 $Y=3$

0010 $\rightarrow$ 商 $Q=2$

0000 $\rightarrow$ 余数 $R=0$



- ① 先左移，空出的位用于上商
 - ② 上商
 - 由控制逻辑根据中间余数与除数的符号组合 来决定本轮做减法 or 加法（同号减，异号加）
 - 再根据ALU运算结果（新余数）的符号位来决定上商为 1 还是 0
 - 新余数与老余数相比，符号不变 $\rightarrow$ 上商 1；符号变 $\rightarrow$ 上商 0
 - ③ 计数器 $C_n --$ ，当计数器 $C_n=0$ 时，除法运算结束
- n轮处理

例2：带符号整数除法

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

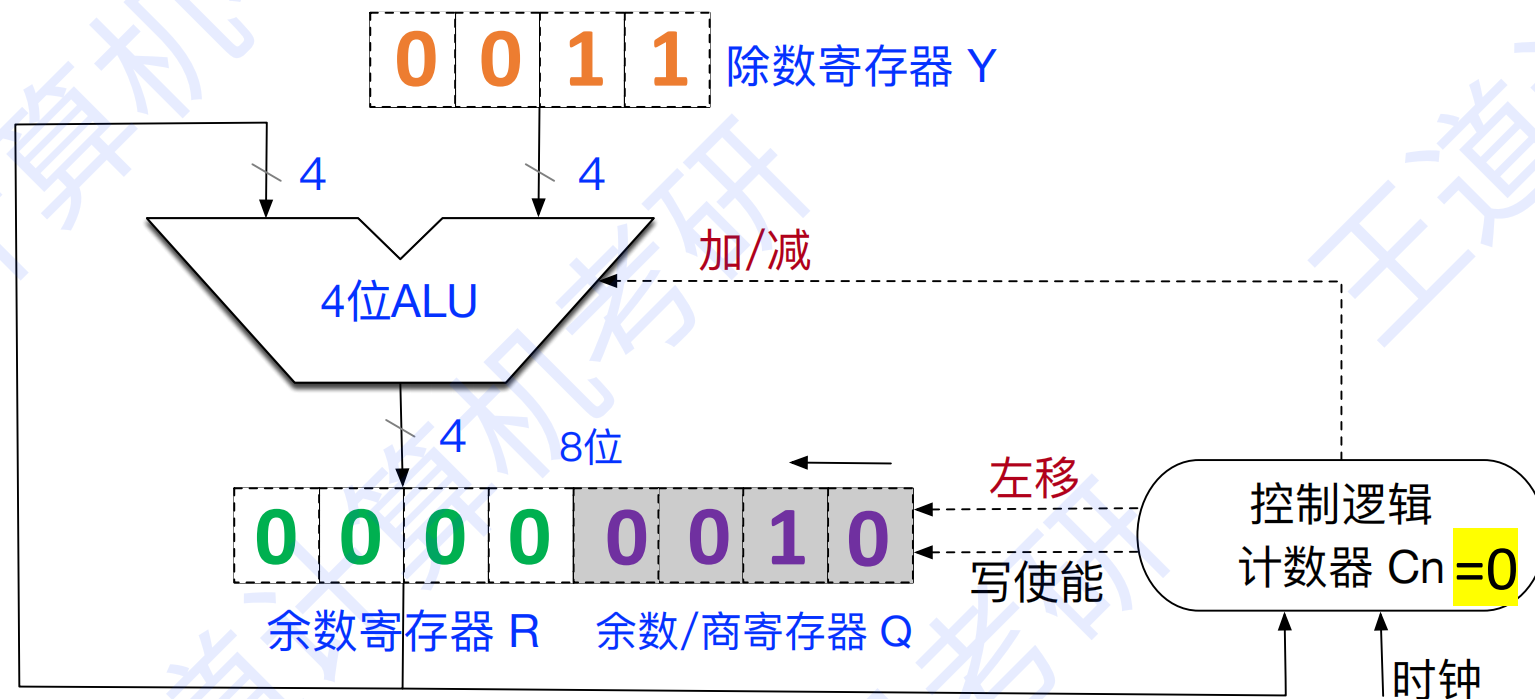
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

0110 $\rightarrow$ 被除数 $X=6$

0011 $\rightarrow$ 除数 $Y=3$

0010 $\rightarrow$ 商 $Q=2$

0000 $\rightarrow$ 余数 $R=0$



注：“除法电路的基本原理”部分，能够结合手算推演除法电路初始状态、结束状态，大致了解中间需要几轮运算即可。

结束——当计数器 $C_n=0$ 时，除法运算结束。 n bit 寄存器 **【R】** 保存余数、 n bit 寄存器 **【Q】** 保存商

例3：特殊情况——可直接得到运算结果

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n \text{ bit} \div n \text{ bit}$

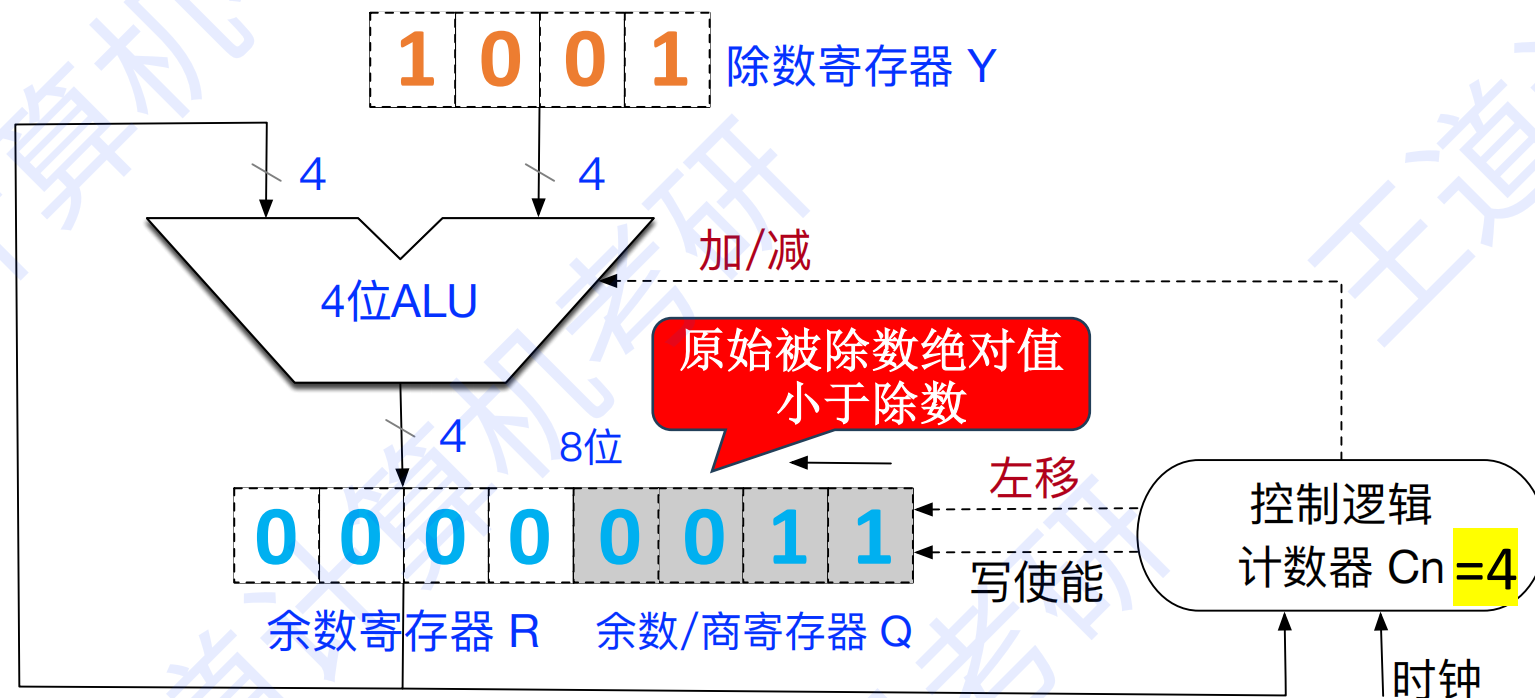
该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数

0011 → 被除数 $X=3$

1001 → 除数 $Y=-7$

0000 → 商 $Q=0$

0011 → 余数 $R=3$



特殊情况判断

可直接得到除法运算结果? —— 如果 $|被除数| < |除数|$ ，则 商=0，余数=被除数，除法器不必再执行

除数是否为0? —— 如果除数为0，发生“除数为0”异常，停止除法运算，调出操作系统的异常处理程序

是否可能发生溢出? —— 考试要点：对于补码的单精度除法（即 $n \text{ bit} \div n \text{ bit}$ ，得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数），仅“绝对值最大的负数 $\div -1$ ”，才有可能发生“商溢出”异常

例3：特殊情况——可直接得到运算结果

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n \text{ bit} \div n \text{ bit}$

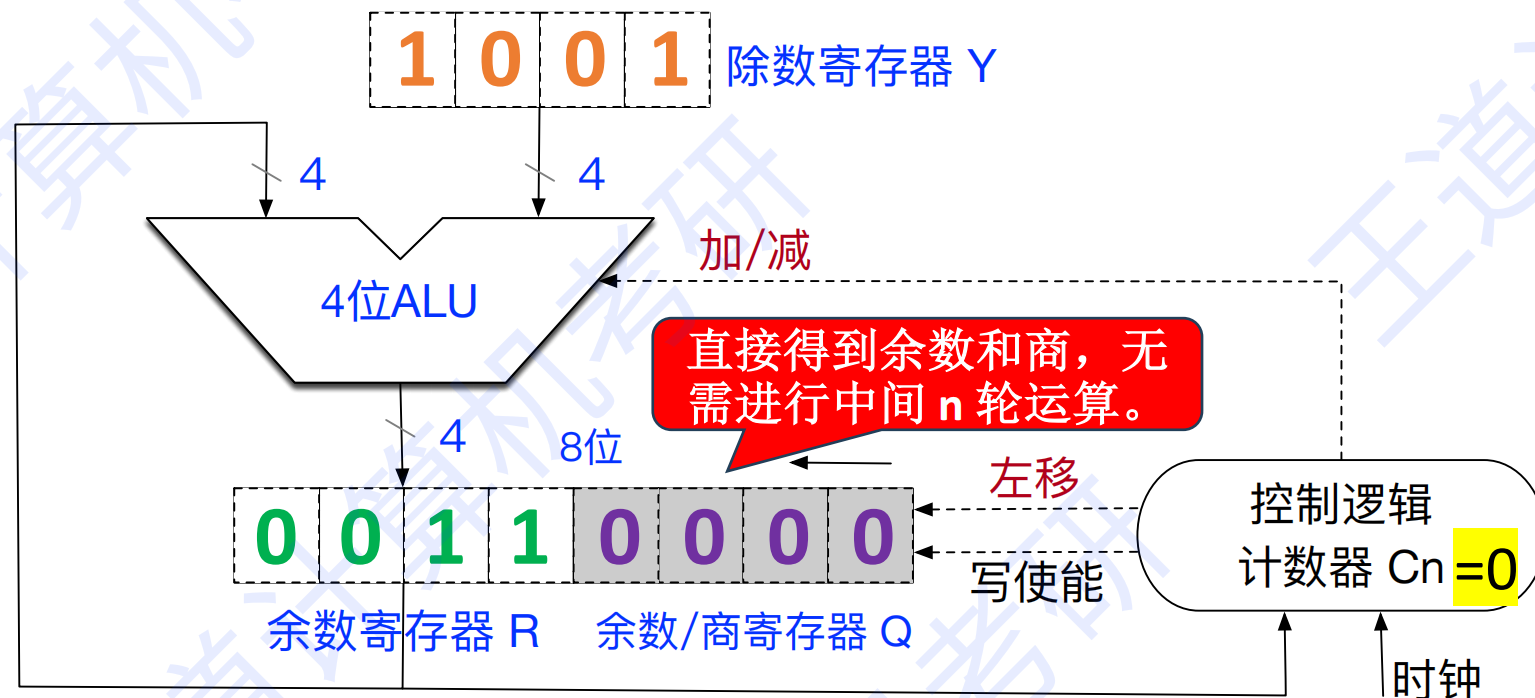
该除法器支持 $2n \text{ bit} \div n \text{ bit}$
最终得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数

0011 → 被除数 $X=3$

1001 → 除数 $Y=-7$

0000 → 商 $Q=0$

0011 → 余数 $R=3$



特殊情况判断

可直接得到除法运算结果？——如果 $|被除数| < |除数|$ ，则 商=0，余数=被除数，除法器不必再执行

除数是否为0？——如果除数为0，发生“除数为0”异常，停止除法运算，调出操作系统的异常处理程序

是否可能发生溢出？——考试要点：对于补码的单精度除法（即 $n \text{ bit} \div n \text{ bit}$ ，得到 $n \text{ bit}$ 商、 $n \text{ bit}$ 余数），仅“绝对值最大的负数 $\div -1$ ”，才有可能发生“商溢出”异常

例4：特殊情况——除以0异常

若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

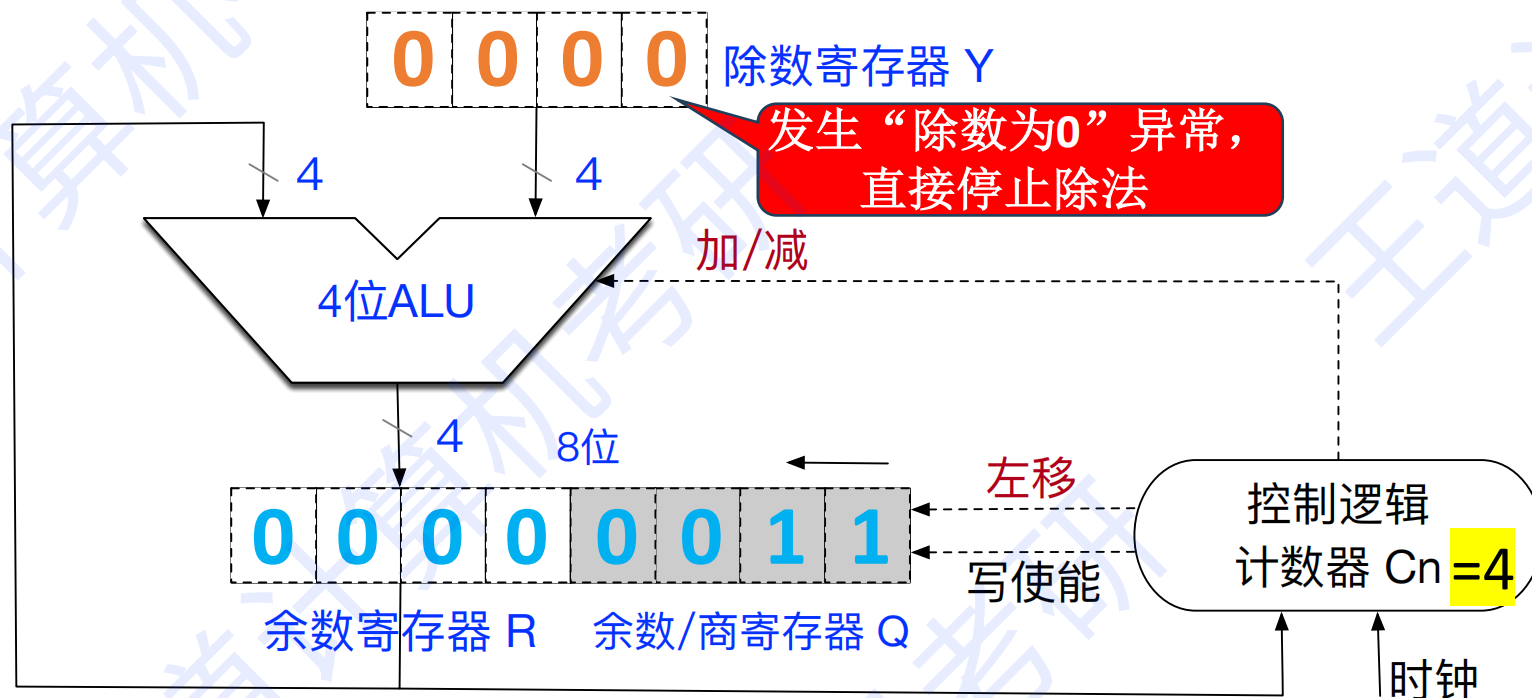
该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

0011 $\rightarrow$ 被除数 $X=3$

0000 $\rightarrow$ 除数 $Y=0$

NULL $\rightarrow$ 商 $Q=NULL$

NULL $\rightarrow$ 余数 $R=NULL$



特殊情况判断

可直接得到除法运算结果？——如果 $|被除数| < |除数|$ ，则 商=0，余数=被除数，除法器不必再执行

除数是否为0？——如果除数为0，发生“除数为0”异常，停止除法运算，调出操作系统的异常处理程序

是否可能发生溢出？——考试要点：对于补码的单精度除法（即 n bit $\div$ n bit，得到 n bit 商、 n bit 余数），仅“绝对值最大的负数 $\div -1$ ”，才有可能发生“商溢出”异常

例5：特殊情况——除法溢出异常

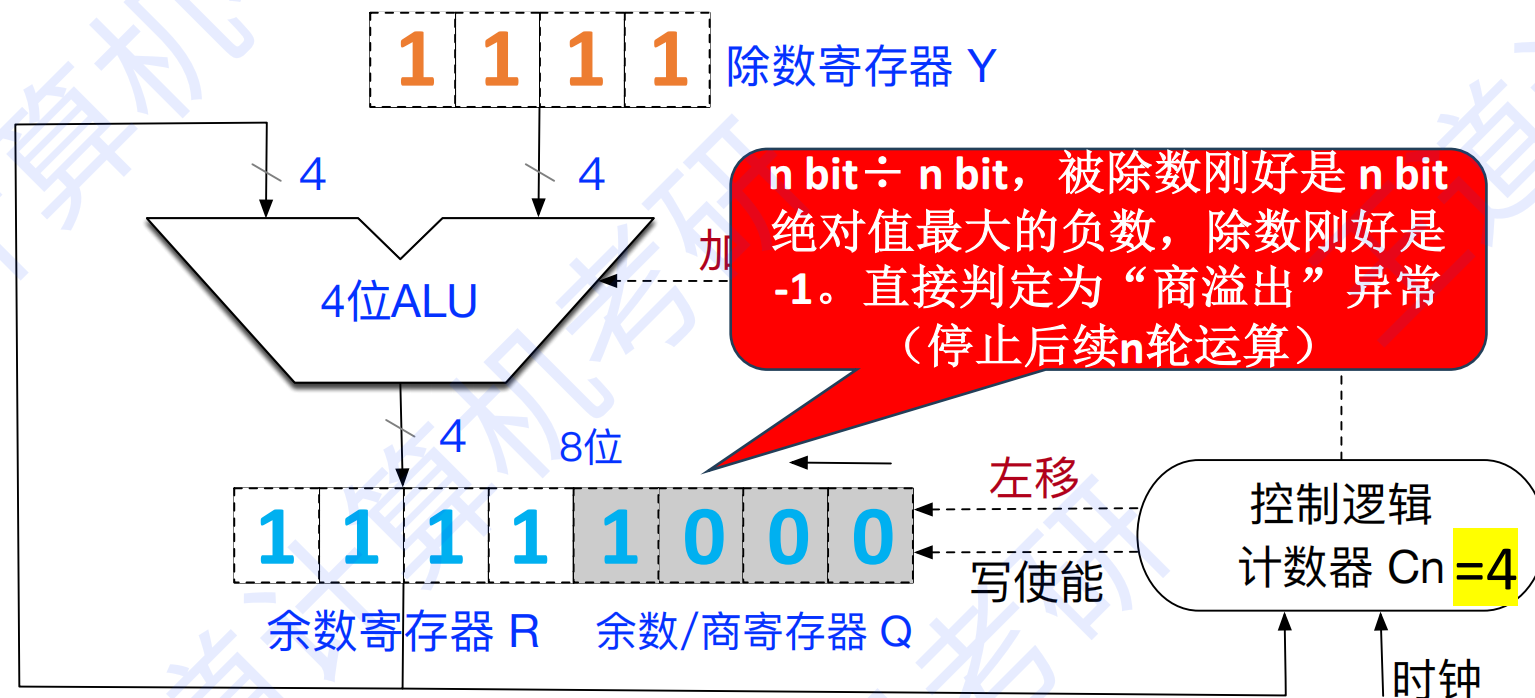
若被除数不足 $2n$ bit，需要扩展为 $2n$ bit，统一为 $2n$ bit $\div$ n bit

该除法器支持 $2n$ bit $\div$ n bit
最终得到 n bit 商、 n bit 余数

1000 $\rightarrow$ 被除数 $X=-8$

1111 $\rightarrow$ 除数 $Y=-1$

$\rightarrow$ 商 $Q=8$ ，溢出4bit补码范围



特殊情况判断

可直接得到除法运算结果？——如果 $|被除数| < |除数|$ ，则 商=0，余数=被除数，除法器不必再执行

除数是否为0？——如果除数为0，发生“除数为0”异常，停止除法运算，调出操作系统的异常处理程序

是否可能发生溢出？——考试要点：对于补码的单精度除法（即 n bit $\div$ n bit，得到 n bit 商、 n bit 余数），仅“绝对值最大的负数 $\div -1$ ”，才有可能发生“商溢出”异常